LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 5 – REKURSIF

HAIDAR RAFI ARSHIA
109082530022
KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Rekursif adalah metode dalam pembuatan program di mana subprogram (fungsi atau prosedur) dipanggil secara terus-menerus sampai menyelesaikan suatu masalah. Rekursif merupakan pendekatan yang berusaha menyelesaikan suatu masalah yang cukup besar melalui penguraian menjadi sub-masalah yang lebih kecil namun mempunyai sifat yang sama dengan masalah induk tersebut.

Pendekatan rekursif sering digunakan sebagai alternatif yang efektif untuk menggantikan kontrol iteratif dengan menggunakan tumpukan pemanggilan subprogram. Untuk mencegah program memanggil diri sendiri sampai ke tak terhingga, setiap algoritma rekursif pasti memerlukan dua elemen utama, yakni base case dan recursive case. Base case merupakan kondisi henti yang umumnya menggunakan struktur kontrol percabangan (if-then) untuk menetapkan ketika program harus berakhir.

Jika base case bernilai true, program akan berhenti memanggil dirinya sendiri dan mulai menyelesaikan tumpukan perintah yang lainnya. Tanpa adanya base case yang jelas, program rekursif akan terus berjalan sampai mengakibatkan crash sistem. Ada dua tahapan utama dalam penggunaan rekursif, yaitu tahap forward dan backward. Forward adalah proses dimana subprogram dipanggil secara berulang-ulang sampai menemukan base case.

Setelah base case dicapai, program akan mengulangi proses backward, dimana ia akan kembali ke subprogram pemanggil dan menjalankan instruksi yang tersisa sampai ke panggilan awal. Penting untuk dipahami urutan pemanggilan instruksi, karena adanya perubahan dalam pengaturan baris perintah.

B. Guided

1. **Guided1.go**

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Print("Masukkan angka: ")
    fmt.Scan(&n)
    baris(n)
}

func baris(bilangan int) {
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Println(bilangan)
        baris(bilangan - 1)
    }
}
```

Output

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run "d:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia\week5_Modul5\Guided\Guided
1.go"
Masukkan angka: 20
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
```

Penjelasan:

Prosedur yang digunakan oleh program ini disebut prosedur baris untuk menampilkan urutan angka menurun, di mana setiap panggilan mencetak angka bulat saat ini dan kemudian memanggil dirinya sendiri menggunakan nilai baru dari angka – 1. Proses ini akan terus berlanjut hingga kita mencapai kasus dasar kita yaitu angka == 1, yang berarti hanya mencetak 1.

2. Guided2.go

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n-1)
    }
}
```

Output :

```
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run "d:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia\week5_Modul5\Guided\Guided
2.go"
10
55
```

Penjelasan:

Program ini memanfaatkan fungsi penjumlahan yang mengembalikan nilai integer untuk menghitung total akumulasi angka dari 1 sampai **n**. Di dalam fungsi tersebut, jika nilai **n** belum mencapai *base-case* n == 1, maka program akan melakukan *return* hasil tambah antara nilai **n** saat ini dengan hasil pemanggilan fungsi penjumlahan(n-1) secara berulang hingga semua angka terakumulasi.

3. **Guided3.go**

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}

func pangkat(n int) int {
    if n == 0 {
        return 1
    } else {
        return 2 * pangkat(n-1)
    }
}
```

Output :

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Code

PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run "d:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia\Week5_Modul5\Guided\Guided
3.go"
20
1048576
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>
```

Penjelasan:

Fungsi Program ini digunakan untuk melakukan perhitungan dari hasil pangkat dengan base 2 melalui penggunaan fungsi pangkat dengan parameter **n**, yang mana merupakan nilai dari pangkat tersebut. Proses ini akan dimulai dari base-case jika n == 0 yang akan menghasilkan angka 1, sementara recursive-case akan terus memperkalikan angka 2 dengan hasil pangkat(n - 1).

C. Unguided

1. Soal Unguided 1

DeretFibonacci.go

Jawaban :

```go
package main

import "fmt"

func fibonacci(n int) int {

    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    }

    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    var n int
    fmt.Print("Masukkan n: ")
    fmt.Scan(&n)

    for i := 0; i <= n; i++ {
        fmt.Printf("%d ", fibonacci(i))
    }
}
```

Output :

```
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run "d:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia\Week5_Modul5\Unguided\DeretFibonacci.go"
Masukkan n: 10
0 1 1 2 3 5 8 13 21 34 55
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>
```

Penjelasan :

Program ini memiliki fungsi dalam program ini dikembangkan dengan menggunakan metode rekursi yang menentukan nilai suku ke-n dalam barisan Fibonacci berdasarkan rumus S_{n} = S_{n-1} + S_{n-2}. Terdapat dua kasus dasar atau base case dalam fungsi ini, yakni jika nilai n sama dengan 0 maka nilai yang dihasilkan adalah 0, serta jika n sama dengan 1 maka nilai yang dihasilkan adalah 1. Dengan menggunakan metode rekursif, setiap kali fungsi dipanggil dengan suku ke-n akan memunculkan fungsi baru untuk dua suku sebelumnya hingga mencapai nilai basis tersebut.

2. Soal Unguided 2

FaktorBilangan.go

Jawaban :

```go
package main

import "fmt"

func cariFaktor(n, i int) {

    if i > n {
        return
    }

    if n%i == 0 {
        fmt.Printf("%d ", i)
    }

    cariFaktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan N: ")
    fmt.Scan(&n)

    cariFaktor(n, 1)
}
```

Output :

```
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
orBilangan.go"
Masukkan N: 5
1 5
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
orBilangan.go"
Masukkan N: 12
1 2 3 4 6 12
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>
```

Penjelasan :

Program ini adalah menampilkan semua angka bulat positif yang bisa membagi N secara urut dari angka 1 sampai N menggunakan pendekatan atau metode rekursif. Dasarnya, program ini mengecek modulus atau sisa bagi dalam setiap panggilan fungsi dimana jika angka pembagi berhasil membagi N tanpa sisa, angka tersebut ditulis sebagai faktor. Proses ini diulang berulang kali dengan mengubah atau menambah nilai angka pembagi sampai menemukan base case, yaitu angka N.

3. Soal Unguided 3

BarisanBilanganGanjil.go

Jawaban :

```go
package main

import "fmt"

func ganjil(n, current int) {

    if current > n {
        return
    }

    if current%2 != 0 {
        fmt.Printf("%d ", current)
    }

    ganjil(n, current+1)
}

func main() {
    var n int
    fmt.Print("Masukkan N: ")
    fmt.Scan(&n)

    ganjil(n, 1)
}
```

Output :

```
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
sanBilanganGanjil.go"
Masukkan N: 20
1 3 5 7 9 11 13 15 17 19
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>
```

Penjelasan :

Algoritma program ini dirancang untuk mem-print bilangan ganjil dari 1 sampai nilai maksimal N yang ditentukan oleh user. Dalam fungsi rekursif ini, bilangan yang diberikan dievaluasi untuk mengetahui apakah itu adalah bilangan ganjil, dan jika iya, maka di-print; selanjutnya, fungsi tersebut dipanggil lagi secara berurutan, tetapi dengan penambahan bilangan (seperti 2 atau 1 dengan kondisi tertentu). Ini akan diulangi secara berurutan dalam kasus rekursif sampai bilangan tersebut melebihi atau menjadi sama dengan N.

D. Kesimpulan

Pengertian rekursif merupakan suatu pendekatan dimana subprogram dipanggil secara berulang oleh dirinya sendiri guna mendefinisikan sub-masalah yang sama dengan bagian besar masalah tersebut. Untuk dapat melakukannya tanpa menghasilkan error infinite loop, setiap algoritme harus memuat base-case sebagai kondisi stop dan recursive-case sebagai kondisi pemanggilan diri sendiri. Kehilangan kedua kondisi tersebut membuat program terus berlanjut hingga membuat error sistem.

Alur kerjanya terdiri dari dua fase, yakni forward (maju menuju base-case) dan backward (keluar kembali ke pemanggil sebelumnya setelah tercapainya base-case). Aplikasi metode ini sangat beragam mulai dari pemanggilan barisan angka, melakukan operasi penjumlahan akumulatif, hingga perhitungan rumus matematika seperti perpangkatan dan deret Fibonacci. Pemahaman alur ini penting karena urutan penempatan kode sangat menentukan hasil akhir keluaran program.